

Trabajo Práctico Integrador:

Alumnas

Alterio Micaela, Ambrosio Laura, Grille Agustina, Guardia Karen

Comisión 7: Alterio, Ambrosio, Guardia, Grille

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación II

Docente Titular

Cinthia Rigoni

Docente Tutor

Luciano Chirolí

17 de Noviembre de 2025

Tabla de contenido

Objetivos	2
Introducción	2
Integrantes y roles	2
Elección de dominio y justificación	2
Diseño y Modelado UML	3
Persistencia, estructura de la Base de Datos y DAO	4
Transacciones	4
Validaciones y reglas del negocio	4
Orden de operaciones y transacciones	5
Validaciones y reglas del negocio en la capa Service	6
Pruebas realizadas	7
Conclusiones y mejoras futuras	8
Herramientas utilizadas	8

Objetivo general de aprendizaje

Desarrollar una aplicación Java modular que implemente una relación 1→1 unidireccional entre las clases Libro y FichaBibliográfica, aplicando el patrón DAO para la persistencia de datos en MySQL mediante JDBC, con el fin de afianzar los conocimientos de programación orientada a objetos, diseño por capas y manejo de transacciones.

Objetivos específicos

- ❖ **Diseñar y modelar** las clases Libro y FichaBibliográfica utilizando UML, reflejando una relación unidireccional 1→1, asegurando la integridad referencial en la base de datos y una estructura coherente entre el modelo lógico y físico.
- ❖ **Implementar la capa de persistencia** aplicando el patrón DAO para separar la lógica de acceso a datos de la lógica de negocio, utilizando PreparedStatements, manejo de conexiones externas y control de transacciones (commit/rollback).
- ❖ **Desarrollar una interfaz de consola funcional** que permita realizar operaciones CRUD completas sobre ambas entidades, con manejo adecuado de excepciones, validaciones y mensajes claros para el usuario.

Introducción

El presente trabajo final integrador tiene como objetivo consolidar los conocimientos adquiridos en la materia Programación 2, mediante la creación de una aplicación Java basada en una relación 1→1 unidireccional entre las clases Libro y FichaBibliográfica. La aplicación sigue una arquitectura por capas, donde se implementan las entidades, los DAOs, los servicios y un menú de consola que permite interactuar con la base de datos MySQL a través de JDBC.

Este proyecto busca fortalecer el manejo de conceptos como la programación orientada a objetos, la persistencia de datos, las transacciones, y la aplicación de patrones de diseño, promoviendo además el trabajo colaborativo y la calidad del código.

Integrantes y roles

Micaela Alterio: Estructura y setup del proyecto + UML + entidades principales + edición del video.

Karen Guardia: Entidad base y entidades complementarias + reglas del dominio + README

Laura Ambrosio: Persistencia de datos + acceso controlado a la base + estructuración del informe.

Agustina Grille: Capa de Servicios (Service Layer) + AppMenu

Elección del dominio y justificación.

Se eligió el dominio Libro → FichaBibliográfica por su claridad conceptual y aplicabilidad en sistemas de gestión bibliográfica o de librerías. La relación principal es de tipo uno a uno (1:1), donde cada libro posee una única ficha bibliográfica que detalla su información técnica (ISBN, idioma, editorial, sinopsis, etc.). Esta relación se definió como unidireccional, es decir, el libro conoce a su ficha, pero la ficha no hace referencia al libro, lo cual evita redundancia y mantiene la integridad del modelo.

Para ampliar el alcance del sistema y simular un escenario de uso más realista, se incorporaron las entidades Cliente y Venta. De esta forma, un cliente puede realizar múltiples compras, y cada venta se asocia a un solo libro y un solo cliente. Estas relaciones (N:1) permiten representar transacciones reales y facilitan la aplicación de constraints como claves foráneas y verificaciones de dominio.

Diseño: decisiones clave

En el modelo Libro → FichaBibliográfica se optó por una relación 1:1 unidireccional, donde la entidad Libro posee un campo `id_ficha` como clave foránea única (FK UNIQUE) que referencia a FichaBibliografica. Esta decisión permitió mantener la independencia de las entidades y garantizar que cada libro esté asociado a una sola ficha sin duplicación de registros. Se descartó el uso de una clave primaria compartida (PK compartida) porque habría unido demasiado ambas tablas, limitando la independencia entre las entidades.

Las entidades Venta y Cliente se incorporaron para extender la funcionalidad del sistema hacia un escenario comercial. La tabla ventas contiene claves foráneas hacia libros y clientes, reflejando relaciones N:1 (múltiples ventas pueden referir al mismo libro o cliente). Además, se aplicaron restricciones CHECK para asegurar valores positivos en los campos precio y cantidad.

A nivel de diseño UML, el diagrama de clases refleja una estructura clara:

- ❖ Libro → FichaBibliografica: asociación unidireccional 1:1
- ❖ Venta → Libro: asociación unidireccional N:1.
- ❖ Venta → Cliente: asociación unidireccional N:1.

Este modelo asegura integridad y coherencia, permitiendo mantener la trazabilidad entre los libros, su información técnica y las transacciones asociadas.

Arquitectura por capas

El sistema se desarrolló siguiendo una arquitectura en capas, con una separación definida de funciones que facilita la organización del código y su futura evolución:

- ❖ **Paquete Entities:** contiene las clases que representan las entidades del dominio (Libro, FichaBibliografica, Venta, Cliente). Cada clase define sus atributos, constructores y métodos de acceso (getters y setters).
- ❖ **Paquete DAO:** implementa el acceso a datos. Incluye una interfaz genérica `GenericDAO<T>` con los métodos CRUD comunes, y clases concretas que la implementan para cada entidad. Esta capa se encarga de ejecutar las operaciones SQL mediante conexiones JDBC,

controlando las claves primarias y foráneas.

- ❖ **Paquete Service:** concentra la lógica de negocio. Valida los datos antes de su inserción o actualización y coordina las operaciones entre distintas entidades. Por ejemplo, al registrar una venta, verifica la existencia del libro y del cliente antes de ejecutar el commit.
- ❖ **Paquete Main:** corresponde a la interfaz de usuario. Contiene el menú principal y las opciones interactivas que permiten realizar operaciones CRUD desde consola de forma guiada.

Esta estructura modular mejora la claridad del sistema, reduce el acoplamiento entre componentes y permite incorporar nuevas funcionalidades sin afectar las capas existentes.

Persistencia:

- ❖ **Estructura de la base de datos**
 - **Tablas principales:** libros, fichas bibliográficas, ventas, clientes.
 - Relación 1:1 entre libros y fichas_bibliograficas implementada con clave foránea única en libros.
Relaciones N:1: ventas → libros, ventas → clientes
 - **Constraints utilizadas:** PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK.
 - **Integridad referencial** se aseguró mediante el uso de claves foráneas
- ❖ **DAO y operaciones básicas**
 - Cada DAO (LibroDAO, FichaBibliograficaDAO, etc.) Implementa operaciones **CRUD** usando **PreparedStatement** para evitar inyecciones SQL.
 - Los DAO reciben una **Connection** externa, lo que les permite formar parte de transacciones coordinadas en la capa de servicios.
 - Excepciones manejadas dentro del DAO, pero commit/rollback aún no implementados (capa services).

Transacciones

- ❖ **Validaciones y reglas de negocio**

La integridad y las reglas de negocio se abordaron en tres niveles, asegurando que los datos sean consistentes desde la creación del objeto hasta su persistencia transaccional.

 - Consistencia a nivel de Entidad (Clases Base, Cliente, Venta)
 - Implementación de la Clase Base: Se creó la clase abstracta Base.java para proveer la clave primaria (id: Long) y un control uniforme de la Baja Lógica a través del atributo eliminado (Boolean). Esta estructura asegura que todas las entidades del sistema (incluyendo Libro, FichaBibliografica, Cliente y Venta) soporten el Soft Delete.
 - Reglas de Unicidad en Cliente: La clase Cliente.java define reglas de negocio clave, como la unicidad del email (mapeado a UNIQUE en la BD). Además, se

sobrescribieron los métodos equals() y hashCode() basándose en este campo $\text{\$email}$, garantizando que a nivel de objetos en memoria, dos clientes con el mismo email sean considerados idénticos.

- Relaciones N:1 de Apoyo: La entidad Venta.java maneja las referencias a Libro y Cliente, estableciendo las relaciones N:1 necesarias para simular transacciones.

❖ Chequeos de Formato y Obligatoriedad (Clase Validador)

- Para garantizar que solo datos limpios entren al flujo transaccional, se implementó una clase de utilidad estática Validador.java en el paquete Service que actúa como una barrera de entrada para la capa Service.
- Esta clase centraliza validaciones que no requieren acceso a la BD, lanzando IllegalArgumentException ante fallos. Las validaciones clave proporcionadas son:
- Verificación de campos obligatorios (validarObligatorio) para todas las entidades.
- Validación de formato de correo electrónico (validarFormatoEmail) para la entidad Cliente.
- Validación de valores positivos (validarPositivo) para montos en Venta.

Orquestación de Integridad y Transacciones (Responsabilidad de la Capa Service)

El chequeo de las Reglas de Unicidad en BD y la Regla Cruzada 1 -> 1 son responsabilidad directa de la capa Service. Esta división asegura que la lógica de validación simple esté separada de la compleja lógica de negocio que requiere consultas a la base de datos y control de commit() / rollback().

Orden de operaciones y transacciones

La capa de servicios implementa transacciones reales con **commit** y **rollback**, utilizando el TransactionManager.

TransactionManager

- ❖ startTransaction() → desactiva autocommit.
- ❖ commit() → confirma cambios.
- ❖ rollback() → revierte la transacción ante cualquier error.
- ❖ close() → garantiza rollback si quedó activa.

LibroService

- ❖ Crear libro → commit al final
- ❖ Actualizar libro → commit al final
- ❖ Eliminar libro → commit
- ❖ Si ocurre cualquier excepción → rollback automático

FichaBibliograficaService

- ❖ Insertar ficha → commit

- ❖ Actualizar ficha → commit
- ❖ Eliminar ficha → commit
- ❖ Validaciones de ISBN → pueden disparar excepciones → rollback automático

Relación libro/ficha

Método asignarFichaALibro(idLibro, idFicha):

1. tx.startTransaction()
2. Validación de existencia del libro y de la ficha
3. UPDATE libro con id_ficha
4. tx.commit()

Si algo falla: rollback automático en el close().

Validaciones y reglas de negocio (Capa Service)

Validaciones en LibroService

- ❖ Título obligatorio
- ❖ Autor obligatorio
- ❖ Año opcional, pero si existe debe ser numérico
- ❖ Género opcional
- ❖ Libro debe existir para actualizar o eliminar
- ❖ No se puede asignar ficha inexistente

Validaciones en FichaBibliograficaService

- ❖ Editorial obligatoria
- ❖ ISBN obligatorio
- ❖ Idioma obligatorio
- ❖ Número de páginas debe ser positivo
- ❖ **Regla especial:** No permitir duplicar ISBN
 - Validación en creación
 - Validación en actualización (excluyendo la propia ficha)

Regla cruzada: relación libro ↔ ficha

- ❖ No se asigna una ficha a un libro si no existe
- ❖ El servicio valida y persiste la relación manteniendo integridad referencial

Result Grid

Filter Rows: Edit: Export/Import: Wrap Cell Content:

	id_ficha	editorial	ISBN	idioma	num_paginas	sinopsis	eliminado
▶	50003	UTN	10109992	Español	1000	Agus aprueba programacion 2	0
	NUL	NUL	NUL	NUL	NUL	NUL	NUL

Conclusiones y mejoras futuras

Conclusiones

- ❖ El sistema implementa correctamente CRUD para libros y fichas.
- ❖ La capa de servicios garantiza integridad, validación y consistencia.
- ❖ Las transacciones aseguran que los cambios en la base sean confiables.
- ❖ El AppMenu permite probar todas las funciones de forma simple e intuitiva.
- ❖ Se logró una separación clara entre DAO, servicios y presentación.

Mejoras futuras

- ❖ Implementar interfaz gráfica en Swing o JavaFX.
- ❖ Agregar autenticación de usuarios.
- ❖ Manejar excepciones más específicas.
- ❖ Implementar caching de consultas frecuentes.
- ❖ Permitir exportar listados a CSV o PDF.
- ❖ Agregar más relaciones (clientes, ventas, stock).

Herramientas utilizadas

- ❖ GitHub y GitBash
- ❖ UMLetino
- ❖ NetBeans
- ❖ MySQL Workbench
- ❖ MySQL server 8.4
- ❖ DBeaver
- ❖ ChatGpt:
 - Para consultar cómo representar atributos protected y atributos Static.
 - Para consultar qué tipo de flechas se usan en UMLetino para representar relaciones.
 - Para corrección de errores e implementación de comentarios descriptivos del código.
 - Resolución de dudas en tipos de protección para los métodos.
 - Optimización de la documentación.
 - Medición de tiempo para la construcción de un guión que se mantenga en los 15 minutos de duración del video solicitado para la entrega del TPI.